恒生电子股份有限公司

编码规范 C、C++分则

2007 年 9 月

目录

1 布局结构规范 1
1.1 一般规则 1
1.2 文件头部说明 1
1.3 函数头部说明 1
1.4 头文件结构 2
1.5 源文件结构 2
1.6 MAKEFILE文件结构 2
1.7 目录结构 3
2 书写排版规范 4
2.1 一般规则 4
2.2 空格 4
2.3 空行 5
2.4 对齐 6
2.5 缩行 7
2.6 长行拆分 8
3 命名规范 9
3.1 一般规则 9
3.2 标示符命名 9
3.3 文件命名 11
3.4 其它命名 11
4 语言规范 12
4.1 常量 12
4.2 变量 12
4.3 宏 12
4.4 表达式 12
4.5 语句 13
4.5.1 IF语句 13
4.5.2 CASE语句 14
4.5.3 循环语句 14
4.5.4 GOTO语句 15
4.6 函数 15
4.7 注释 17
5 其它规范 19

注：本文中标注【**规则*-*-***】的条款必须严格执行，标注【**建议*-*-***】的条款为提倡采用。

1 布局结构规范

1.1 一般规则

【规则 1-1-1】每个源程序文件的头部必须包含文件头部说明和修改记录（与头部说明合并）。
【规则 1-1-2】每个函数头部必须包含函数头部说明。
【建议 1-1-1】每个源程序文件的尾部最好有文件结束说明或标志。

1.2 文件头部说明

【规则 1-2-1】源程序的文件头部说明必须包含系统名称、模块名称、文件名称、软件版权、功能说明、系统版本、开发人员、开发时间和修改记录等几个部分，审核人员、审核日期和相关文档为可选项，各部分次序参见《总则》说明；

【规则 1-2-2】源程序的修改记录说明必须包含修改日期、修改人员、修改说明，每次修改为一条记录。

1.3 函数头部说明

【规则 1-3-1】函数的头部说明必须包含函数名称、函数功能、输入参数、输出参数、返回值等几个部分，对环境引用、对环境影响、内部变量说明、内部变量说明和被调函数列表为可选项，各部分次序如下：

函数名称	本函数的名称
函数功能	简述函数的功能
输入参数	列出主要的、不易理解的输入参数的含义和取值范围
输出参数	列出主要的、不易理解的输出参数的含义和取值范围
返回值	函数返回值的含义和取值范围
对环境引用（可选的）	需要引用的系统环境，如全局变量、环境变量等
对环境影响（可选的）	可能会改变的系统环境，如全区变量、环境变量等
内部变量说明(可选的)	本函数内部的重要的变量说明
内部变量说明(可选的)	本函数内部主要调用的函数清单
被调函数列表(可选的)	调用本函数的主要函数清单

1.4 头文件结构

【规则 1-4-1】头文件各部分的书写顺序如下：

文件头部说明	见前
包含的头文件	#include
常量定义	const
宏定义	#define
类型定义	typedef
结构、联合定义	struct 或 union
外部变量声明	extern
全局变量定义	
函数原型声明	

1.5 源文件结构

【规则 1-5-1】源文件各部分的书写顺序如下：

文件头部说明	见前
包含的头文件	#include
常量定义	const
宏定义	#define
类型定义	typedef
结构、联合定义	struct 或 union
外部变量声明	extern
全局变量定义	
函数原型声明	
一般函数	
main 函数	

1.6 Makefile 文件结构

【规则 1-6-1】makefile 文件各部分的书写顺序如下：

系统名称	系统或工程的名字
宏定义	CC=cc
依赖关系	.SUFFIXES: .o .c .c.o: cc -o $*.o
生成项	all: mytest clean
生成规则	mytest: a.o cc $(CC_FLAG)

1.7 目录结构

【规则 1-7-1】在源程序目录下，无用的文件必须移走或删除。

【规则 1-7-2】保持目录结构清晰，头文件与源文件要存放在不同的子目录下，系统公用的头文件与某程序私有的头文件也要分别放在不同的子目录中。

【建议 1-7-1】以下的目录结构可供参考：

```
$HOME/
|-----.profile              PROFILE 文件
|-----bin/                  执行文件目录
|-----etc/                  配置文件目录
|-----log/                  日志文件目录
|-----tmp/                  临时目录
|-----src/                  源文件目录
    |
    |------public/          公共源文件目录
    |       |------api/     公共 API 文件目录
    |       |------c/       公共 C 文件目录
    |       |------ec/      公共 EC 文件目录
    |       |------include/ 公共头文件目录
    |       |------lib/     公共库文件目录
    |       |------libincl/ 公共库文件的头文件目录
    |       |------tm/      临时目录
    |
    |------??????/          某个模块目录
    |       |------c/       C 文件目录
    |       |------ec/      EC 文件目录
    |       |------include/ 头文件目录
    |       |------tmp      临时目录
… …
```

2 书写排版规范

2.1 一般规则

【规则 2-1-1】不允许把多个短语句写在一行中，即一行最多只写一条语句。
【规则 2-1-2】合理使用空格、空行、对齐、缩行、长行拆分等方式格式化源代码，使程序舒展，可读。

2.2 空格

【规则 2-2-1】关键字之后要留空格。象 const、define、case 等关键字之后至少要留一个空格，否则无法辨析关键字。象 if、for、while、switch 等关键字之后应留一个空格再跟左括号‘(’，以突出关键字。
【规则 2-2-2】‘,’之后要至少要留一个空格；
【规则 2-2-3】如果‘;’不是一行的结束符号，其后要留一个空格。
【规则 2-2-4】赋值操作符、比较操作符、算术操作符、逻辑操作符、位域操作符，如“=”、“+=” “>=”、“<=”、“+”、“*”、“%”、“&&”、“||”、“<<”，“^”等二元操作符的前后应当各加一个空格。
【规则 2-2-5】注释前后适当加一个或若干个空格。
【规则 2-2-6】函数名之后紧跟左括号‘(’，中间不要留空格，以与关键字区别。
【规则 2-2-7】‘(’之后和‘)’、‘,’、‘;’之前都不留空格。
【规则 2-2-8】一元操作符如“!”、“~”、“++”、“--”、“*” （指针取值符）、“&”（地址运算符）等前后不得加空格。
【规则 2-2-9】象“[]”、“.”、“->”这类操作符前后不加空格。

提倡的风格	不提倡的风格
if (year >= 2000)	if(year>=2000)
if ((a>=b) && (c<=d))	if(a>=b&&c<=d)
for (i = 0; I < 10; i++)	for(i=0;i<10;i++)
x = a < b ? a : b;	x=a<b?a:b;
void Func1(int x, int y, int z);	void Func1 (int x,int y,int z);
int *x = &y;	int * x = & y;
array[5] = 0;	array [5] = 0;
b->data	b -> data
/* 我的注释 */	/*我的注释*/

2.3 空行

【规则 2-3-1】每个函数定义结束之后都要加一个或若干个空行。

例如：

```
空行
void Function1(…)
{
  …
}
空行
void Function2(…)
{
  …
}
空行
void Function3(…)
{
  …
}
空行
```

【规则 2-3-2】在一个函数体内，变量定义与函数语句之间要加空行。

例如：

```
void Function1(…)
{
    int n;
    空行
    statement1;
    …
}
```

【规则 2-3-3】逻辑上密切相关的语句之间不加空行，其它地方应加空行分隔。

例如：

```
 空行
while (condition)
{
  statement1;
  空行
  if (condition)
  {
      statement2;
  }
  else
```

```
    {
        statement3;
    }
    空行
    statement4;
  }
```

2.4 对齐

【规则 2-4-1】程序的分界符‘**{**’和‘**}**’永远都单独成行并且位于同一列，同时与引用它们的语句左对齐。

例如：

提倡的的风格	不提倡的风格

void Function(int x)
{
… program code
}
	void Function(int x){
	… program code
	}

if (condition)
{
… program code
}
else
{
… program code
}
	if (condition){
	… program code
	}
	else{
	… program code
	}

for (initialization; condition; update)
{
… program code
}
	for (initialization;condition; update){
	… program code
	}

while (condition)
{
… program code
}
	while (condition){
	… program code
	}

do
{
… program code
}
while (condition);
	do{
	… program code
	}while (condition);

【规则 2-4-2】CASE 语句语的 case 部分与 switch 的‘{’列对齐。

例如：

提倡的风格	不提倡的风格

switch (variable)
{
case value1 :
…
break;
case value2 :
…
break;
…
default :
…
break;
}
	switch (variable)
	{
	case value1 : …
	break;
	case value2 : …
	break;
	…
	default : …
	break;
	}

2.5 缩行

【规则 2-5-1】用缩行显示程序结构，使排版整齐，缩进量统一使用 4 个字符。

【建议 2-5-1】最好用 4 个空格而不是 Tab 来实现缩行，这样无论在什么环境上看到的效果都是相同的。

【规则 2-5-2】在函数体的开始、结构/联合的定义、枚举的定义以及循环、判断等语句中的代码都要采用缩行。

例如：

提倡的风格	不提倡的风格

void Function(int x)
{
… program code
}
	void Function(int x)
	{
	… program code
	}

struct tagMyStruct
{
int a;
int b;
int c;
};
	struct tagMyStruct
	{
	int a;
	int b;
	int c;
	};

if (condition)
{
… program code
}
else
{
… program code
}
	if (condition)
	{
	… program code
	}
	else
	{
	… program code
	}

【规则 2-5-3】同层次的代码在同层次的缩进层上。

2.6 长行拆分

【规则 2-6-1】代码行最大长度宜控制在 80 个字符以内，较长的语句、表达式等要分成多行书写。

【规则 2-6-2】长表达式要在低优先级操作符处划分新行，操作符放在新行之首（以便突出操作符）。拆分出的新行要进行适当的缩进，使排版整齐，语句可读。

例如：

```
if ((very_longer_variable1 >= very_longer_variable12)
    && (very_longer_variable3 <= very_longer_variable14)
    && (very_longer_variable5 <= very_longer_variable16))
{
    dosomething();
}
```

```
for (very_longer_initialization;
     very_longer_condition;
     very_longer_update)
{
    dosomething();
}
```

【规则 2-6-3】如果函数中的参数较长，则要进行适当的划分。

例如：

```
void Function(float very_longer_var1,
              float very_longer_var2,
              float very_longer_var3)
```

3 命名规范

3.1 一般规则

【规则 3-1-1】命名应当直观且可以拼读，可望文知意，便于记忆和阅读。

标识符最好采用英文单词或其组合，避免使用拼音。程序中的英文单词一般不会太复杂，用词应当准确。例如不要把 CurrentValue 写成 NowValue。

【规则 3-1-2】命名的长度应当符合“min-length && max-information”原则。

C 是一种简洁的语言，命名也应该是简洁的。例如变量名 maxval 就比 maxValueUntilOverflow 好用。标识符的长度一般不要过长，较长的单词可通过去掉“元音”形成缩写。

【建议 3-1-1】尽量避免名字中出现数字编号，如 Value1,Value2 等，除非逻辑上的确需要编号。这是为了防止程序员偷懒，不肯为命名动脑筋而导致产生无意义的名字（因为用数字编号最省事）。

3.2 标示符命名

【规则 3-2-1】程序中不得出现仅靠大小写区分的相似的标识符。

例如：

```
int  x,  X;          变量 x 与 X 容易混淆
void foo(int x);     函数 foo 与 F00 容易混淆
void F00(float x);
```

【规则 3-2-2】局部变量不得与全局变量重名，尽管两者的作用域不同而不会发生语法错误，但会使人误解。

【规则 3-2-3】变量的名字应当使用“名词”或者“形容词＋名词”。

例如：

```
float   value;
float   oldValue;
float   newValue;
```

【规则 3-2-4】全局函数的名字应当使用“动词”或者“动词＋名词”（动宾词组）。各单词连写，词首字母大写

例如：DrawBox();

【规则 3-2-5】用正确的反义词组命名具有互斥意义的变量或相反动作的函数等。

例如：

```
int minValue;
int maxValue;

int SetValue(…);
int GetValue(…);
```

【规则 3-2-6】所有宏定义、typedef 定义类型、枚举常数、常量全用大写字母命名，用下划线分割单词。

例如：

```
const int MAX_LENGTH = 100;
#define FILE_PATH "/usr/tmp"
```

【规则 3-2-7】静态变量加前缀 s_（表示 static）。

例如：

```
void Init(…)
{
    static int s_initValue;
…
}
```

【规则 3-2-8】全局变量加前缀 g_（表示 global）。

例如：

```
int g_iCount;
```

【规则 3-2-9】数据库宿主变量加前缀 m。

例如：

```
EXEC SQL begin declare section;
    int miCount;
EXEC SQL end declare section;
```

【规则 3-2-10】考虑到习惯性问题，局部变量中可采用通用的命名方式，如 n、i 作为循环变量，p、q 作为指针等。

【建议 3-2-1】对于全局变量和重要的局部变量可使用匈牙利命名法，即用小写字母的前缀表示变量的类型，前缀的下一个字母用大写。

具体见下表：

类型	前缀	类型	前缀
char	c	unsigned	u
int	i	指针	p
long	l	数组	n
float	f	结构	S
double	d	联合	U
string	s		

例如：

```
int niArray[ARRAY_SIZE];
unsigned char ucByte;
char sName[20 ];
char *psName = sName;
struct MyStruct *pSTemp;
```

3.3 文件命名

【规则 3-3-1】项目组应采用相同的风格命名源文件，全小写风格：组成文件名的单词全采用小写字母，单词间用下划线分隔，避免取一些比较通俗的文件名，如：main.c，public.c 等；首字母大写风格：组成文件名的单词的首字母大写；

3.4 其它命名

【规则 3-4-1】数据库对象命名约定采用“项目缩写+数据库类型+版本号”的形式。(注：这里所指的版本号是指数据库结构的版本号，此版本号可能与系统的版本号不一致)。
例如：股票质押系统英文简称为 hssims， 其当前数据库名为 simscur10， 历史库名为 simshis10，表名取其英文名。
【规则 3-4-2】为保证移植性，嵌入式 SQL 语句标识统一用大写的“EXEC SQL”，不允许用“$”；变量引用统一采用“：”，不允许用“$”。

4 语言规范

4.1 常量

【规则 4-1-1】需要对外公开的常量放在头文件中，不需要对外公开的常量放在定义文件的头部。为便于管理，可以把不同模块的常量集中存放在一个公共的头文件中。

【规则 4-1-2】如果某一常量与其它常量密切相关，应在定义中包含这种关系，而不应给出一些孤立的值。

例如：

```
const   float   RADIUS = 100.0;
const   float   DIAMETER = RADIUS * 2;
```

4.2 变量

【规则 4-2-1】声明变量的同时对变量进行初始化，严禁使用未经初始化的变量。

【规则 4-2-2】变量类型的说明应注意机器字长以及应用项目中该变量得取值范围。

4.3 宏

【规则 4-3-1】宏体和参数都要加上括号。

例如：

正确的宏定义	错误的宏定义
#define MAX(a, b) ((a) > (b) ? (a) : (b))	#define MAX(a, b) a > b ? a : b

【规则 4-3-2】为杜绝可能带来的副作用，尽量避免采用函数宏。

【规则 4-3-3】由于 const 常量有数据类型，编译器可以对此进行类型安全检查，所以不要用宏来定义常量，而用 const。

4.4 表达式

【规则 4-4-1】如果代码行中的运算符比较多，用括号确定表达式的操作顺序，避免使用默认的优先级。

例如：

```
leap_year = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
```

【规则 4-4-2】不要编写太复杂的复合表达式。

例如：

```
i = a >= b && c < d && c + f <= g + h;
```

复合表达式过于复杂

【规则 4-4-3】不要有多用途的复合表达式。

例如：

```
d = (a = b + c) + r;
```

该表达式既求 a 值又求 d 值。应该拆分为两个独立的语句：

```
a = b + c;
d = a + r;
```

【规则 4-4-4】尽量避免含有否定运算的条件表达式。

例如：

如：`if (!(num >= 10))`

应改为：`if (num < 10)`

4.5 语句

4.5.1 IF 语句

【规则 4-5-1】if 语句本身自占一行，执行语句不得紧跟其后。不论执行语句有多少都要加{}。

例如：

提倡的风格	不提倡的风格

`if (width < height)`
`{`
`  dosomething();`
`}`
	`if (width < height) dosomething();`

【规则 4-5-2】不可将浮点变量用“==”或“！=”与任何数字比较。

千万要留意，无论是 float 还是 double 类型的变量，都有精度限制。所以一定要避免将浮点变量用“==”或“！=”与数字比较，应该设法转化成“>=”或“<=”形式。

假设浮点变量的名字为 x，应当将

```
if (x == 0.0)
```

转化为

```
if ((x >= -EPSINON) && (x <= EPSINON))
```

其中 EPSINON 是允许的误差（即精度）。

【规则 4-5-3】应当将指针变量用“==”或“！=”与 NULL 比较。

指针变量的零值是“空”（记为 NULL）。尽管 NULL 的值与 0 相同，但是两者意义不同。假设指针变量的名字为 p，它与零值比较的标准 if 语句如下：

```
if (p == NULL)    p 与 NULL 显式比较，强调 p 是指针变量
if (p != NULL)
```

不要写成

```
if (p == 0)       容易让人误解 p 是整型变量
if (p != 0)
```

或者

```
if (p)            容易让人误解 p 是布尔变量
if (!p)
```

【建议 4-5-1】变量与常数做相等比较时，将常数写在前面，变量写在后面。

例如：用 if (0 == Value) 来代替 if (Value == 0)，这样是为了当程序员将 == 误写为=，编译器马上能够发现报错。

4.5.2 CASE 语句

【规则 4-5-4】每个 case 语句的结尾绝对不要忘了加 break，否则将导致多个分支重叠(除非有意使多个分支重叠)。

【规则 4-5-5】不要忘记最后那个 default 分支。即使程序真的不需要 default 处理，也应该保留语句：

```
default :
    break;
```

4.5.3 循环语句

【规则 4-5-6】对于没有循环体的循环语句，使用 NULL;来代替空语句。

例如：

提倡的风格	不提倡的风格
while (…) NULL;	while (…);
for (…) NULL;	for (…);

【建议 4-5-2】在多重循环中，如果有可能，应当将最长的循环放在最内层，最短的循环放在最外层，以减少 CPU 跨切循环层的次数。

例如：

长循环在最内层，效率高	长循环在最外层，效率低

for (col=0; col<5; col++)
{
for (row=0; row<100; row++)
{
sum = sum + a[row][col];
}
}
	for (row=0; row<100; row++)
	{
	for (col=0; col<5; col++)
	{
	sum = sum + a[row][col];
	}
	}

【建议 4-5-3】建议 for 语句的循环控制变量的取值采用“半开半闭区间”写法。

半开半闭区间写法和闭区间写法虽然功能是相同，但相比之下，半开半闭区间写法写法更加直观。

半开半闭区间写法	闭区间写法

for (n = 0; n < 10; n++)
{
…
}
	for (n = 0; n <= 9; n++)
	{
	…
	}

4.5.4 GOTO 语句

【规则 4-5-7】少用、慎用 goto 语句。

自从提倡结构化设计以来，goto 就成了有争议的语句。首先，由于 goto 语句可以灵活跳转，如果不加限制，它的确会破坏结构化设计风格；其次，goto 语句经常带来错误或隐患。它可能跳过了变量的初始化、重要的计算等语句，例如：

```
goto state;
int sum = 0;      被 goto 跳过
…
state:
…
```

如果编译器不能发觉此类错误，每用一次 goto 语句都可能留下隐患。

很多人建议废除 goto 语句，以绝后患。但实事求是地说，错误是程序员自己造成的，不是 goto 的过错。goto 语句至少在出错处理中是采用是比较有利的，它能从多重循环体中一下子跳到外面，用不着写很多次的 break 语句。例如

```
{ …
    { …
        { …
            goto error;
        }
    }
}
error:
…
```

4.6 函数

【规则 4-6-1】函数参数定义采用现代风格。

例如：

提倡的风格	不提倡的风格

int main(int argc, char * argv[])
{
…
}
	int main(argc, argv)
	int argc;
	char * argv[];
	{
	…
	}

【规则 4-6-2】参数的书写要完整，不要贪图省事只写参数的类型而省略参数名字。如果函数没有参数，则用 void 填充。

例如：

提倡的风格	不提倡的风格
void SetValue(int width, int height);	void SetValue(int, int);

float GetValue(void);	float GetValue();

【规则 4-6-3】参数命名要恰当，顺序要合理。

例如编写字符串拷贝函数 StringCopy，它有两个参数。如果把参数名字起为 str1 和 str2，例如

```
void StringCopy(char *str1, char *str2);
```

那么我们很难搞清楚究竟是把 str1 拷贝到 str2 中，还是刚好倒过来。

可以把参数名字起得更有意义，如叫 strSource 和 strDestination。这样从名字上就可以看出应该把 strSource 拷贝到 strDestination。

还有一个问题，这两个参数那一个该在前那一个该在后？参数的顺序要遵循程序员的习惯。一般地，应将目的参数放在前面，源参数放在后面。

如果将函数声明为：

```
void StringCopy(char *strSource, char *strDestination);
```

别人在使用时可能会不假思索地写成如下形式：

```
char str[20];
StringCopy(str, “Hello World”);    参数顺序颠倒
```

【规则 4-6-4】如果参数是指针，且仅作输入参数用，则应在类型前加 const，以防止该指针在函数体内被意外修改。

例如：

```
void StringCopy(char *strDestination, const char *strSource);
```

【规则 4-6-5】不要省略返回值的类型，如果函数没有返回值，那么应声明为 void 类型。

【规则 4-6-6】不要将正常值和错误标志混在一起返回。正常值用输出参数获得，而错误标志用返回值返回。

【规则 4-6-7】在函数体的“入口处”，对参数的有效性进行检查。

【规则 4-6-8】return 语句不可返回指向“栈内存”的“指针”，因为该内存在函数体结束时被自动销毁。

例如：

```
char * Func(void)
{
    char str[30];
    …
    return str;
}
```

str 属于局部变量，位于栈内存中，在 Func 结束的时候被释放，所以返回 str 将导致错误。

【建议 4-6-1】函数的功能要单一，不要设计多用途的函数。

【建议 4-6-2】函数体的规模要小，尽量控制在 200 行代码之内。

【建议 4-6-3】相同的输入应当产生相同的输出。尽量避免函数带有“记忆”功能。

带有“记忆”功能的函数，其行为可能是不可预测的，因为它的行为可能取决于某种“记忆状态”。这样的函数既不易理解又不利于测试和维护。在 C 语言中，函数的 static 局部变量是函数的“记忆”存储器。建议尽量少用 static 局部变量，除非必需。

【建议 4-6-4】避免函数有太多的参数，参数个数尽量控制在 5 个以内。如果参数太多，在使用时容易将参数类型或顺序搞错。

【建议 4-6-5】尽量不要使用类型和数目不确定的参数。

C 标准库函数 printf 是采用不确定参数的典型代表，其原型为：

```
int printf(const chat *format[, argument]…);
```

这种风格的函数在编译时丧失了严格的类型安全检查。

【建议 4-6-6】有时候函数原本不需要返回值，但为了增加灵活性如支持链式表达，可以附加返回值。

例如字符串拷贝函数 strcpy 的原型：

```
char *strcpy(char *strDest, const char *strSrc);
```

strcpy 函数将 strSrc 拷贝至输出参数 strDest 中，同时函数的返回值又是 strDest。这样做并非多此一举，可以获得如下灵活性：

```
char str[20];
int  length = strlen(strcpy(str, “Hello World”) );
```

【建议 4-6-7】不仅要检查输入参数的有效性，还要检查通过其它途径进入函数体内的变量的有效性，例如全局变量、文件句柄等。

4.7 注释

【建议 4-7-1】建议采用注释文档提取工具（如 doxygen），并按工具标准对函数、类、变理的声明进行注释。

【规则 4-7-1】注释应当准确、易懂，防止有二义性。错误的注释不但无益反而有害。

【规则 4-7-2】边写代码边注释，修改代码同时修改相应的注释，以保证注释与代码的一致性。不再有用的注释要及时删除。

【规则 4-7-3】注释是对代码的“提示”，而不是文档。程序中的注释不可喧宾夺主，注释太多了会让人眼花缭乱。

【规则 4-7-4】一目了然的语句不加注释。

例如：

```
i++;    /* i 加 1 */   多余的注释
```

【规则 4-7-5】对于全局数据（全局变量、常量定义等）必须要加注释。

【规则 4-7-6】注释统一用 C 语言的/* 和*/， 不允许用 C++的//；

【规则 4-7-7】注释采用中文，尽量避免在注释中使用缩写，特别是不常用缩写。

【规则 4-7-8】注释的位置应与被描述的代码相邻，可以与语句在同一行，也可以在上行，但不可放在下方。同一结构中不同域的注释要对齐。

【规则 4-7-9】当代码比较长，特别是有多重嵌套时，应当在一些段落的结束处加注释，便于阅读。

例如：

```
if (…)
{
    …
    while (…)
```

```
    {
      …
    } /* while 语句结束 */
    …
} /* if 语句结束 */
```

5 其它规范

【规则 5-1-1】防止引用已经释放的内存空间。

例如：

```
char *p = (char *) malloc(100);
strcpy(p, “hello”);
free(p);        /* p 所指的内存被释放，但是 p 所指的地址仍然不变 */
…
if (p != NULL)  /* 没有起到防错作用 */
{
   strcpy(p, “world”);    /* 出错 */
}
```

【规则 5-1-2】对于运行期会被频繁调用的代码中动态申请的内存，应进行内存泄露检查。

常见错误有：

(1)同一函数内进行内存申请与释放时，其间的有多个 retrun 点，有的 return 点忘了进行内存释放，导致内存泄露。

(2)出现野指针，如:

```
char *p = (char *) malloc(100);
…
p = other();  /*原来所指内存再也没有指针引用*/
…
free(p);
```

(3)重分配内存失败,如:

```
char *p = (char *) malloc(100);
…
p = (char *) realloc(p,iNewSize);  /*realloc 失败时，p 被置为 NULL，原有内存丢失*/
if (p) …
free(p);
```

【规则 5-1-3】malloc、free 与 new、delete 必须配对使用，即 malloc 分配的内存不能用 delete 释放，new 的内存不能用 free 释放。

【规则 5-1-4】进行内存、字符串拷贝操作（如 strcpy、sprintf、memcpy）时，检查并防止内存越界。

【规则 5-1-5】跨平台 C++编程时，字符串操作函数（如 strlen,strcpy）要求所操作的字符串不能为 NULL(若为 NULL，在非 windows 平台下会直接异常退出)，即函数原型中参数类型为 const char *，则默认要求调用者提供非 NULL 的实参；此规则也适用所有用 const 修饰的指针。

【规则 5-1-6】函数中申请的（为打开文件而使用的）文件句柄，在函数退出前要关闭。

【规则 5-1-7】打开编译器的所有告警开关后对程序进行编译。

【建议 5-1-1】内存分配与释放，互斥锁的加锁与释放尽量保同一层次的代码内完成，以便检查是否成对。

【建议 5-1-2】移植性建议

为保证能顺利地移植到各种 Unix/Linux 系统，建议：

自动检测并定义操作系统和编译器标识，以便处理不兼容的代码，一般采用编译器内置宏定义、环境变量及手工指定环境宏等方式来区分不同的环境。

识别项目中使用的可能存在平台差异的代码，并进行必要的独立封装：

1、 字节序与字边界敏感的应用
2、 进程与线程相关应用
3、 IPC 应用
   (1)消息传递（管道、FIFO、消息队列）
   (2)同步（互斥锁、条件变量、读写锁、信号量）
   (3)共享内存（无名，有名）
   (4)远程调用（solaris门，sun RPC）
4、 Socket 应用
5、 文件系统与动态加载动态库（共享库）
6、 模板与 STL 中的平台差异
7、 Makefile 的平台差异
8、 不同平台下的不同数据库系统的访问

在对整数所占字节数有要求的地方使用预定义整数类型，规则如下：

u（表示是否有符号）＋ int ＋ 所占字节数，如：

typedef unsigned char uint8
typedef unsigned short uint16
typedef unsigned long uint32
typedef long int32

【修订记录】

批准日期	修订说明	编制部门
2007/09/12	初稿通过评审	EPG
	修改 logo	EPG

删除的内容: 2007/11/30